

**SEKOLAH TINGGI TEKNOLOGI
PRATAMA ADI PROGRAM STUDI
TEKNIK INFORMATIKA**

**LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2**

Modul Praktikum : Implementasi Dictionary dan Set untuk Optimasi Data
Nama : Sony Iman Kurnia
NIM : 552010125017
Kelas : Teknik Informatika
Dosen Pengampu : Yudi Herdiana, S.T., M.T.
Tanggal Pengumpulan : 05 – Juni – 2026



**SEKOLAH TINGGI TEKNOLOGI PRATAMA
ADI
PROGRAM STUDI TEKNIK INFORMATIKA**

2026

DAFTAR ISI

BAB I TUJUAN PRAKTIKUM	3
BAB II DASAR TEORI	3
BAB III ALAT DAN BAHAN	4
BAB IV LANGKAH KERJA	4
BAB V HASIL EKSEKUSI	4
5.1 Latihan 1 – Pencarian Menggunakan List	4
5.2 Latihan 2 – Pencarian Menggunakan Dictionary	5
5.3 Latihan 3 – Menghilangkan Duplikasi dengan Set.....	6
5.4 Latihan 4 – Integrasi Modular + Dictionary	7
5.5 Perbandingan singkat antara list dan dictionary	8
5.6 Analisis kompleksitas operasi lookup	8
5.7 Refleksi Praktikum	8
BAB VI KESIMPULAN	9
DAFTAR PUSTAKA	9

BAB I TUJUAN PRAKTIKUM

Tujuan praktikum ini adalah membuat program pengelolaan data mahasiswa menggunakan dictionary sebagai struktur penyimpanan data serta menerapkan konsep modular programming dalam Python. Praktikum ini juga bertujuan untuk memahami proses penambahan, pencarian, dan penampilan data secara efisien menggunakan dictionary.

- Memahami konsep algoritma dan IPO (Input, Process, Output)
- Memahami penggunaan fungsi modular dalam program Python
- Memahami penggunaan dictionary untuk menyimpan data mahasiswa (NIM dan nilai)
- Memahami cara menambah, menampilkan, dan mencari data dalam dictionary
- Memahami penggunaan perulangan dan percabangan pada menu interaktif
- Memahami penggunaan exception handling (try-except) untuk menangani kesalahan input
- Menganalisis perbedaan list dan dictionary serta kompleksitas operasi lookup pada dictionary.

BAB II DASAR TEORI

- **Hashing**
Hashing adalah teknik yang digunakan untuk mengubah data menjadi nilai hash sehingga proses penyimpanan dan pencarian data dapat dilakukan dengan cepat.
- **Lookup Dictionary**
Dictionary merupakan struktur data yang menyimpan pasangan key-value. Data dapat dicari berdasarkan key tanpa harus memeriksa seluruh elemen.
- **Kompleksitas $O(1)$ Average**
Operasi pencarian (lookup) pada dictionary memiliki kompleksitas rata-rata $O(1)$, yang berarti waktu pencarian relatif konstan meskipun jumlah data bertambah.
- **Pencarian Key**
Pada dictionary, pencarian dilakukan menggunakan key sebagai identitas unik sehingga akses data menjadi lebih efisien dibandingkan pencarian berurutan.
- **Set untuk Menghilangkan Duplikasi**
Set adalah struktur data yang hanya menyimpan elemen unik. Jika terdapat data yang sama, set secara otomatis menghapus duplikasi tersebut.
- **Optimasi dengan Mengganti Linear Search**
Penggunaan dictionary dapat mengoptimalkan proses pencarian karena tidak perlu melakukan linear search ($O(n)$) pada seluruh data. Dengan hashing, pencarian dapat dilakukan lebih cepat dengan kompleksitas rata-rata $O(1)$.

BAB III ALAT DAN BAHAN

1. Laptop
2. Visual Studio Code
3. Word
4. Python
5. ChatGPT
6. Kuota

BAB IV LANGKAH KERJA

1. Buka laptop
2. Buka apk Visual Studio Code
3. Ketik syntax dengan teliti sesuai dengan tugas yang diberikan
4. Simpan file
5. Buka program / running program di extension phyton
6. Buat laporan
7. Selesai

BAB V HASIL EKSEKUSI

5.1 Latihan 1 – Pencarian Menggunakan List

1. Syntax

```
1 # Sony Iman Kurnia
2 # 552010125017
3 # 30 - Mei - 2026
4
5 import os
6
7 # Membersihkan terminal
8 os.system('cls' if os.name == 'nt' else 'clear')
9
10 data = ["A001", "A002", "A003", "A004"]
11
12 ulang = "y"
13
14 while ulang.lower() == "y":
15     target = input("Masukkan NIM yang dicari: ")
16
17     count = 0
18     found = False
19
20     for item in data:
21         count += 1
22         if item == target:
23             found = True
24             break
25
26     print("Ditemukan:", found)
27     print("Jumlah langkah:", count)
28
29     ulang = input("\nCari data lagi? (y/t): ")
30
31 print("Program selesai.")
```

2. Pengujian

```
Masukkan NIM yang dicari: A001
Ditemukan: True
Jumlah langkah: 1

Cari data lagi? (y/t): y
Masukkan NIM yang dicari: A004
Ditemukan: True
Jumlah langkah: 4

Cari data lagi? (y/t): y
Masukkan NIM yang dicari: A005
Ditemukan: False
Jumlah langkah: 4

Cari data lagi? (y/t):
```

3. Analisis Kompleksitas: $O(n)$

Program ini menggunakan metode Linear Search, yaitu mencari data dengan memeriksa elemen satu per satu dari awal hingga data ditemukan.

Kompleksitasnya adalah $O(n)$ karena pada kondisi terburuk program harus memeriksa seluruh data dalam list. Semakin banyak data yang disimpan, semakin banyak pula langkah yang diperlukan untuk proses pencarian. Oleh karena itu, Linear Search kurang efisien untuk data yang berjumlah besar.

5.2 Latihan 2 – Pencarian Menggunakan Dictionary

1. Syntax

```
1 # Sony Iman Kurnia
2 # 552010125017
3 # 30 - Mei - 2026
4
5 import os
6
7 # Membersihkan terminal
8 os.system('cls' if os.name == 'nt' else 'clear')
9
10 data = {
11     "A001": 80,
12     "A002": 75,
13     "A003": 90,
14     "A004": 85
15 }
16
17 ulang = "y"
18
19 while ulang.lower() == "y":
20     target = input("Masukkan NIM yang dicari: ")
21
22     if target in data:
23         print("Nilai:", data[target])
24     else:
25         print("Tidak ditemukan")
26
27     ulang = input("\nCari data lagi? (y/t): ")
28
29 print("Program selesai.")
```

2. Pengujian

```
Masukkan NIM yang dicari: A001
Nilai: 80

Cari data lagi? (y/t): y
Masukkan NIM yang dicari: A004
Nilai: 85

Cari data lagi? (y/t): y
Masukkan NIM yang dicari: A005
Tidak ditemukan

Cari data lagi? (y/t):
```

3. Analisis kompleksitas rata-rata: $O(1)$

Program ini menggunakan dictionary untuk menyimpan data mahasiswa. Proses pencarian dilakukan berdasarkan key (NIM), sehingga data dapat ditemukan secara langsung tanpa memeriksa seluruh elemen satu per satu. Oleh karena itu, kompleksitas rata-rata pencarian pada dictionary adalah $O(1)$. Hal ini membuat proses lookup lebih cepat dan efisien dibandingkan pencarian pada list yang memiliki kompleksitas $O(n)$.

5.3 Latihan 3 – Menghilangkan Duplikasi dengan Set

1. Syntax

```
1 # Sony Iman Kurnia
2 # 552010125017
3 # 30 - Mei - 2026
4
5 import os
6
7 # Membersihkan terminal hanya saat program dijalankan
8 os.system('cls' if os.name == 'nt' else 'clear')
9
10 data = [10, 20, 20, 30, 40, 40, 50]
11
12 while True:
13     print("\n=== PROGRAM SET PYTHON ===")
14     print("1. Tampilkan Data")
15     print("2. Tambah Data")
16     print("3. Tampilkan Data Unik")
17     print("4. Hitung Jumlah Data Unik")
18     print("5. Keluar")
19
20     pilihan = input("Pilih menu: ")
21
22     if pilihan == "1":
23         print("Data awal :", data)
24
25     elif pilihan == "2":
26         try:
27             nilai = int(input("Masukkan nilai baru: "))
28             data.append(nilai)
29             print("Data berhasil ditambahkan.")
30         except ValueError:
31             print("Input harus berupa angka!")
32
33     elif pilihan == "3":
34         unik = set(data)
35         print("Data unik :", unik)
36
37     elif pilihan == "4":
38         unik = set(data)
39         print("Jumlah data unik :", len(unik))
40
41     elif pilihan == "5":
42         print("Program selesai.")
43         break
44
45     else:
46         print("Pilihan tidak valid!")
```

2. Pengujian

```
=== PROGRAM SET PYTHON ===
1. Tampilkan Data
2. Tambah Data
3. Tampilkan Data Unik
4. Hitung Jumlah Data Unik
5. Keluar
Pilih menu: 1
Data awal : [10, 20, 20, 30, 40, 40, 50]

=== PROGRAM SET PYTHON ===
1. Tampilkan Data
2. Tambah Data
3. Tampilkan Data Unik
4. Hitung Jumlah Data Unik
5. Keluar
Pilih menu: 2
Masukkan nilai baru: 50
Data berhasil ditambahkan.

=== PROGRAM SET PYTHON ===
1. Tampilkan Data
2. Tambah Data
3. Tampilkan Data Unik
4. Hitung Jumlah Data Unik
5. Keluar
Pilih menu: 1
Data awal : [10, 20, 20, 30, 40, 40, 50, 50]

=== PROGRAM SET PYTHON ===
1. Tampilkan Data
2. Tambah Data
3. Tampilkan Data Unik
4. Hitung Jumlah Data Unik
5. Keluar
Pilih menu: 3
Data unik : {40, 10, 50, 20, 30}

=== PROGRAM SET PYTHON ===
1. Tampilkan Data
2. Tambah Data
3. Tampilkan Data Unik
4. Hitung Jumlah Data Unik
5. Keluar
Pilih menu: 4
Jumlah data unik : 5

=== PROGRAM SET PYTHON ===
1. Tampilkan Data
2. Tambah Data
3. Tampilkan Data Unik
4. Hitung Jumlah Data Unik
5. Keluar
Pilih menu: 5
Program selesai.
```

3. Analisis kompleksitas

Program ini menggunakan list dan set untuk mengelola data. Operasi penambahan data ke dalam list dengan `append()` memiliki kompleksitas $O(1)$ karena data ditambahkan langsung di akhir list. Sementara itu, proses mengubah list menjadi set dengan `set(data)` memiliki kompleksitas $O(n)$ karena program harus memeriksa setiap elemen dalam list untuk menghilangkan data yang duplikat. Oleh karena itu, menu menampilkan data unik dan menghitung jumlah data unik memiliki kompleksitas $O(n)$. Semakin banyak data yang disimpan, semakin banyak elemen yang harus diproses saat pembentukan set.

5.4 Latihan 4 – Integrasi Modular + Dictionary

1. Syntax

```
1 # Sony Iman Kurnia
2 # 552818125817
3 # 30 - Mei - 2026
4
5 import os
6
7 # Membersihkan terminal
8 os.system('cls' if os.name == 'nt' else 'clear')
9
10
11 def tambah_data(data):
12     nim = input("Masukkan NIM: ")
13
14     try:
15         nilai = float(input("Masukkan nilai: "))
16         data[nim] = nilai
17         print("Data berhasil ditambahkan.")
18     except ValueError:
19         print("Nilai harus berupa angka.")
20
21
22 def tampilkan_data(data):
23     if data:
24         print("\n=== DICTIONARY DATA MAHASISWA ===")
25         for nim, nilai in data.items():
26             print(f"NIM: {nim} | Nilai: {nilai}")
27     else:
28         print("Belum ada data.")
29
30
31 def cari_data(data):
32     nim = input("Cari NIM: ")
33
34     if nim in data:
35         print("Nilai:", data[nim])
36     else:
37         print("Tidak ditemukan.")
38
39
40 def jumlah_unik(data):
41     nilai_unik = set(data.values())
42     print("Nilai unik:", nilai_unik)
43     print("Jumlah nilai unik:", len(nilai_unik))
44
45
46 def info_dictionary(data):
47     print("\n=== INFORMASI DICTIONARY ===")
48     print("Jumlah data:", len(data))
49     print("Key (NIM):", list(data.keys()))
50     print("Value (Nilai):", list(data.values()))
51
52
53 def menu():
54     data = {}
55
56     while True:
57         print("\n=== MENU ===")
58         print("1. Tambah Data")
59         print("2. Tampilkan Data")
60         print("3. Cari Data")
61         print("4. Jumlah Nilai Unik (Set)")
62         print("5. Informasi Dictionary")
63         print("6. Keluar")
64
65         pilih = input("Pilih menu: ")
66
67         if pilih == "1":
68             tambah_data(data)
69
70         elif pilih == "2":
71             tampilkan_data(data)
72
73         elif pilih == "3":
74             cari_data(data)
75
76         elif pilih == "4":
77             jumlah_unik(data)
78
79         elif pilih == "5":
80             info_dictionary(data)
81
82         elif pilih == "6":
83             print("Program selesai.")
84             break
85
86         else:
87             print("Pilihan tidak valid.")
88
89
90 menu()
```

2. Pengujian

```
=== MENU ===
1. Tambah Data
2. Tampilkan Data
3. Cari Data
4. Jumlah Nilai Unik (Set)
5. Informasi Dictionary
6. Keluar
Pilih menu: 1
Masukkan NIM: A001
Masukkan nilai: 90
Data berhasil ditambahkan.

=== MENU ===
1. Tambah Data
2. Tampilkan Data
3. Cari Data
4. Jumlah Nilai Unik (Set)
5. Informasi Dictionary
6. Keluar
Pilih menu: 1
Masukkan NIM: A002
Masukkan nilai: 80
Data berhasil ditambahkan.

=== MENU ===
1. Tambah Data
2. Tampilkan Data
3. Cari Data
4. Jumlah Nilai Unik (Set)
5. Informasi Dictionary
6. Keluar
Pilih menu: 1
Masukkan NIM: A001
Masukkan nilai: 90
Data berhasil ditambahkan.
```

```
=== MENU ===
1. Tambah Data
2. Tampilkan Data
3. Cari Data
4. Jumlah Nilai Unik (Set)
5. Informasi Dictionary
6. Keluar
Pilih menu: 2

=== DICTIONARY DATA MAHASISWA ===
NIM: A001 | Nilai: 90.0
NIM: A002 | Nilai: 80.0

=== MENU ===
1. Tambah Data
2. Tampilkan Data
3. Cari Data
4. Jumlah Nilai Unik (Set)
5. Informasi Dictionary
6. Keluar
Pilih menu: 3
Cari NIM: A002
Nilai: 80.0

=== MENU ===
1. Tambah Data
2. Tampilkan Data
3. Cari Data
4. Jumlah Nilai Unik (Set)
5. Informasi Dictionary
6. Keluar
Pilih menu: 4
Nilai unik: {80.0, 90.0}
Jumlah nilai unik: 2
```

```
=== MENU ===
1. Tambah Data
2. Tampilkan Data
3. Cari Data
4. Jumlah Nilai Unik (Set)
5. Informasi Dictionary
6. Keluar
Pilih menu: 5

=== INFORMASI DICTIONARY ===
Jumlah data: 2
Key (NIM): ['A001', 'A002']
Value (Nilai): [90.0, 80.0]

=== MENU ===
1. Tambah Data
2. Tampilkan Data
3. Cari Data
4. Jumlah Nilai Unik (Set)
5. Informasi Dictionary
6. Keluar
Pilih menu: 6
Program selesai.
```

3. Analisis Kompleksitas

Program ini menggunakan dictionary untuk menyimpan data mahasiswa. Operasi penambahan data (`data[nim] = nilai`) dan pencarian data (`if nim in data`) memiliki kompleksitas rata-rata $O(1)$ karena dictionary menggunakan teknik hashing. Hal ini membuat proses penyimpanan dan pencarian data dapat dilakukan dengan cepat tanpa harus memeriksa seluruh elemen. Sementara itu, proses menampilkan seluruh data dan membuat set dari nilai (`set(data.values())`) memiliki kompleksitas $O(n)$ karena harus mengakses semua data yang tersimpan. Secara keseluruhan, penggunaan dictionary membuat program lebih efisien terutama pada operasi pencarian data berdasarkan NIM.

5.5 Perbandingan singkat antara list dan dictionary

List digunakan untuk menyimpan kumpulan data secara berurutan dan diakses menggunakan indeks. Dictionary menyimpan data dalam bentuk pasangan key dan value. Pada list, pencarian data biasanya dilakukan dengan menelusuri elemen satu per satu. Sedangkan pada dictionary, data dapat dicari langsung menggunakan key. Dictionary lebih cocok digunakan untuk data yang memiliki identitas unik seperti NIM mahasiswa. Sementara itu, list lebih cocok digunakan untuk menyimpan data yang hanya membutuhkan urutan elemen. Oleh karena itu, dictionary umumnya lebih efisien untuk proses pencarian data berdasarkan key.

5.6 Analisis kompleksitas operasi lookup

Operasi lookup pada dictionary memiliki kompleksitas rata-rata $O(1)$ karena menggunakan struktur hash table. Artinya, waktu pencarian relatif tetap meskipun jumlah data bertambah. Sebaliknya, pencarian pada list memiliki kompleksitas $O(n)$ karena harus memeriksa elemen satu per satu hingga data ditemukan. Oleh karena itu, dictionary lebih efisien untuk pencarian data berdasarkan key seperti NIM mahasiswa. Penggunaan dictionary sangat sesuai untuk aplikasi yang sering melakukan operasi pencarian data.

5.7 Refleksi Praktikum

1. Mengapa dictionary lebih cepat dari list untuk pencarian?

Jawab : Dictionary lebih cepat karena menggunakan hashing, sehingga data dapat dicari langsung berdasarkan key tanpa memeriksa semua elemen. Sedangkan pada list, pencarian biasanya dilakukan satu per satu dari awal sampai data ditemukan. Oleh karena itu, pencarian pada dictionary umumnya lebih efisien dibandingkan list.

2. Apakah dictionary selalu $O(1)$?

Jawab: Tidak selalu. Pada kondisi normal, pencarian pada dictionary memiliki kompleksitas rata-rata $O(1)$. Namun, pada kasus tertentu seperti terjadi banyak collision pada hash, kompleksitasnya bisa menurun menjadi $O(n)$. Meskipun begitu, kasus tersebut jarang terjadi sehingga dictionary tetap dianggap sangat cepat untuk pencarian data.

3. Kapan set lebih tepat digunakan dibanding list?

Jawab : Set lebih tepat digunakan ketika kita hanya membutuhkan data yang unik dan tidak ingin ada nilai yang duplikat. Selain itu, set juga lebih efisien untuk memeriksa apakah suatu data ada di dalam kumpulan data. Jika urutan data tidak terlalu penting dan yang dibutuhkan adalah keunikan data, maka set lebih cocok digunakan daripada list.

BAB VI KESIMPULAN

Berdasarkan praktikum yang dilakukan, dapat disimpulkan bahwa dictionary memudahkan penyimpanan dan pencarian data berdasarkan key dengan kompleksitas rata-rata $O(1)$. Sementara itu, set digunakan untuk menghilangkan data duplikat dan memperoleh data yang unik. Penggunaan dictionary lebih efisien dibandingkan linear search pada list yang memiliki kompleksitas $O(n)$. Selain itu, penerapan fungsi modular membuat program lebih terstruktur dan mudah dikelola.

DAFTAR PUSTAKA

- Ramalho, L. (2022). *Fluent Python* (2nd ed.). O'Reilly Media.
- Downey, A. B. (2015). *Think Python: How to Think Like a Computer Scientist* (2nd ed.). O'Reilly Media.
- Sweigart, A. (2019). *Automate the Boring Stuff with Python* (2nd ed.). No Starch Press.
- Python Software Foundation. (2025). *Python Documentation*. <https://docs.python.org/3/>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms*. MIT Press.
- Yudi Herdiana. *Alpro 2: Implementasi Dictionary dan Set untuk Optimasi Data* (File PDF Materi Perkuliahan).